

PROJECT DOCUMENTATION — JUNE 2026

PlotTeaFiles Studio

A local AI-powered pipeline that turns plain story scripts into cinematic short-form videos — automatically.

June 8, 2026

Build Session

7 Major

Upgrades

8

Challenges Solved

Full Stack

Front + Back

01 — OVERVIEW

What is PlotTeaFiles Studio?

PlotTeaFiles Studio is a locally-run automation pipeline that converts a plain English story script into a fully produced vertical video — complete with voiceover, word-level captions, AI-generated cinematic images, and a polished MP4 output suitable for Instagram Reels, YouTube Shorts, or TikTok.

The project was originally scaffolded by an AI coding assistant (Codex) as a CLI tool called **PlotPipe**. The June 2026 session transformed it into a full-stack product with a web-based Studio UI, live progress tracking, model selection, and a complete visual redesign.

The Pipeline — Step by Step

1

Script Input

User pastes a plain English story into the Studio UI. GPT converts it to a structured StoryScript JSON with segments, dialogue, and mood.

2

Voiceover Generation

OpenAI TTS (gpt-4o-mini-tts) or ElevenLabs generates a per-segment narration MP3 from the script text.

3

Caption Transcription

OpenAI Whisper transcribes each audio file into word-level timestamps — used for animated karaoke-style captions in the final video.

4

Image Generation

OpenAI gpt-image-2 generates cinematic 1024×1536 portrait images for each shot in each segment. Multiple images per segment create visual variety during narration.

5

Remotion Render

Remotion (React-based video engine) assembles all assets — audio, images, captions — and renders the final MP4 at full quality.

6

Output & Download

The completed MP4 is saved to `out/studio/` and `plotteafiles/`, and a download link appears in the Studio UI.

02 — TECHNOLOGY

The Tech Stack

Every component in the pipeline was chosen to run entirely locally on a single machine — no external infrastructure required beyond API calls.

RUNTIME

Node.js 18+ / TypeScript

WEB UI

Next.js 16 (App Router)

VIDEO RENDERING

Remotion 4.x

CLI FRAMEWORK

Commander.js

IMAGE GENERATION

OpenAI gpt-image-2 / 1.5 / 1

TTS / VOICEOVER

OpenAI gpt-4o-mini-tts

CAPTIONS

OpenAI Whisper (word timestamps)

SCRIPT PROCESSING

OpenAI GPT (text → JSON)

ALTERNATIVE IMAGE (PARKED)

fal.ai FLUX Pro (coded, zero balance)

PLATFORM

Windows 11, PowerShell

Key Architecture Decision: Local Spawn Pattern

The Studio web UI does not call the pipeline directly. Instead, it spawns the PlotPipe CLI as a child process using Node.js `spawn()`. This means:

- The full pipeline code doesn't need to run inside Next.js
- Stdout/stderr from the CLI is piped back to both the terminal and the progress tracker
- Jobs can be cancelled by killing the child process

- The same CLI can be used standalone (no web UI needed)

03 — SESSION WORK

7 Upgrades Made in This Session

The June 8 session began with reconstructing a prior session's uncommitted changes (context was lost), then continued with a full wave of new upgrades.

1

FLUX Pro Integration (fal.ai)

Added fal-ai/flux-pro as a third image provider. Uses queue-based API: POST to submit → poll status_url until COMPLETED → download. Falls back to OpenAI on failure. Fully coded and wired through the entire stack but parked due to zero fal.ai balance.

2

Caption Style Overhaul

Active word switched from yellow + scale transform to purple highlight background (#7C3AED).

Font: 68px → 74px. Phrase window: 7 words → 5 words (tighter, more focused).

WebkitTextStroke on inactive words only. Padding and gap tightened.

3

Upgrade to gpt-image-2

After confirming gpt-image-2 and gpt-image-1.5 exist (post knowledge cutoff), upgraded the default model from gpt-image-1-mini to gpt-image-2. Quality default changed from 'low' to 'high' throughout the pipeline.

4

Remove Gemini Entirely

Gemini free tier was unstable and not reliably falling back to OpenAI. Removed from imagegen.ts, store.ts, cli.ts, compose.ts, and page.tsx. Simplified the provider type to just 'openai' | 'flux'.

5

Model Selector Dropdown

Added imageModel state and dropdown in the Studio UI with three options: gpt-image-2 (Best), gpt-image-1.5, gpt-image-1. Threaded imageModel through the full stack via --image-model CLI flag.

6

Progress Bar + Time Remaining

Added startedAt timestamp when jobs start. formatTimeRemaining() calculates estimated time left based on elapsed time vs. current progress. UI shows percentage on left, "~Xmin remaining" on right, below a smooth 2px progress bar.



Terminal Logging + UI Redesign

Child process stdout/stderr now piped to terminal, tagged by job ID. Full CSS rewrite: cinematic dark theme (#0a0a0b), orange accent (#f97316), glowing title, editorial panel heads with uppercase letter-spacing, outlined badges, orange-glow buttons.

04 — CHALLENGES & SETBACKS

8 Challenges Overcome

Every build session has its walls. Here's every problem hit, and how each one was solved.

1 — Session Context Lost

User accidentally opened the project folder in a different Claude session, wiping the active conversation. Three prior upgrades were mid-flight with no written record.

Fix: Ran `git diff HEAD` across the 5 modified files. All uncommitted changes were visible in the working tree. Reconstructed the full picture from the diff and briefed the user.

2 — VS Code Python .env Warning

VS Code showed "python.terminal.useEnvFile is disabled" when the `FAL_KEY` was added. User thought this would prevent the key from loading.

Fix: The warning is from the Python VS Code extension — completely irrelevant to Node.js. Next.js reads .env automatically via dotenv. No action needed.

3 — fal.ai Balance Exhausted

`FAL_KEY` was valid and the connection worked, but the API returned "Exhausted balance. Top up at fal.ai/dashboard/billing." FLUX Pro costs ~\$0.05/image.

Fix: Parked FLUX entirely. Kept the full integration code in place — it will work immediately when the user tops up their fal.ai balance.

4 — Wrong npm Command

Running `npm run dev` launched the CLI, not the web server. This caused confusion about why the Studio UI wasn't available.

Fix: Correct commands are `npm run web:dev` (hot reload) or `npm run web:build + npm run web:start` (production, lower RAM). Documented in memory.

5 — Render Appeared Stuck at 58%

Progress bar stopped at 58% during image generation. No terminal logs visible. User suspected the job had frozen or crashed silently.

Fix (immediate): Checked `localhost:3000/api/jobs` — status was still 'running'. Job was alive. Fix (permanent): Added stdout/stderr piping from child process to terminal. Root cause: gpt-image-2 High quality × 15 images = 5–15 minutes. Normal.

6 — Port 3000 EADDRINUSE

After a rebuild, `npm run web:start` failed: "Error: address already in use :::3000". The previous server process hadn't been killed before the rebuild.

Fix: `netstat -ano | findstr :3000` revealed PIDs. Killed them with `Stop-Process -Id <PID> -Force` in PowerShell. Server started cleanly after.

7 — Edit Tool "File Not Read" Errors

Multiple Edit tool calls failed mid-session because the target files hadn't been read into context first. The tool requires a prior Read call before any edit.

Fix: Always Read before Edit. Established as a working discipline for subsequent changes in the session.

8 — Playwright file:// Protocol Blocked

Tried to open mockup HTML files via `file:///` URL in Playwright for the UI comparison exercise. The protocol was blocked.

Fix: Navigated to `about:blank` instead, then injected the full HTML directly via `browser.evaluate` with `document.write()`. Worked perfectly for all 3 mockup directions.

05 — COST ANALYSIS

Image Generation Cost Breakdown

The first tracked render (using gpt-image-1.5 at medium quality, 1 shot per segment, 5 segments) gave us real measured data to plan future renders.

MODEL	QUALITY	IMAGES	MEASURED COST	COST/IMAGE
gpt-image-1.5	Medium	5	\$0.29	~\$0.058
gpt-image-1.5	Medium	10	~\$0.58	~\$0.058
gpt-image-1.5	Medium	15 (3 shots × 5 segments)	~\$0.87	~\$0.058
gpt-image-2	High	15 (3 shots × 5 segments)	~\$1.50–\$2.00 (est.)	~\$0.10–\$0.13

Recommended Strategy

DRAFT / TEST RENDERS

1 shot/segment, gpt-image-1 or 1.5, Medium quality → ~\$0.29 per video

FINAL PRODUCTION

2–3 shots/segment, gpt-image-2, High quality → \$1.50–\$3.00 per video

All images are generated at 1024×1536 (portrait, 9:16 for vertical video). Size is fixed — cost scales by model and quality only.

06 — ROADMAP

What's Still Pending

All planned updates, categorised by priority and blockers.

MAJOR**AI Video Generation**

Generate actual video clips per segment instead of still images. Blocked: no free tier exists for Runway, Kling, Luma, or Sora. Will integrate via fal.ai when user has budget.

MAJOR**FLUX Pro (fal.ai)**

Fully coded and wired through the entire stack. Blocked only by zero fal.ai balance. Will work immediately once account is topped up.

MINOR**Quality Default Fix**

UI quality dropdown still defaults to 'low' in React state. Should default to 'high' to match imagegen.ts. One-line fix in page.tsx.

MINOR**Auto-Start on Boot**

User asked about auto-starting the Studio server. Decision deferred — using manual start to save RAM. Could use pm2 or Windows Task Scheduler in future.

PARKED**Pexels Stock Video**

User rejected this direction — "they don't have much video we can use." Would need AI video generation, not stock footage.

PARKED**Auto-Commit / Git History**

All changes remain as uncommitted working tree modifications. Can be committed to git as a clean milestone commit when ready.

PlotTeaFiles Studio

7

UPGRADES

8

CHALLENGES

5

FILES REBUILT

\$0.06

PER IMAGE

Script → Voiceover → Captions → Cinematic Images → MP4

All local. All automated. All yours.

Built June 2026 · tejrigbe@gmail.com